AI/ML Intern Evaluation Project

Duration: Until next Friday (10 July, 2026)

Prerequisite knowledge assumed: Python, basic ML/AI overview

---

1. Objective

This is a take-home evaluation project. It is designed to test how you **think, research, structure, and ship**, not just whether you can write code. We are looking at:

- How you break down an ambiguous problem into steps
- How you use an LLM as a tool (not as a crutch)
- Code quality, structure, and documentation habits
- Whether your project actually *runs* for someone who isn't you

You are free to use a **free-tier LLM API (Groq, Google AI Studio/Gemini free tier, OpenRouter free models) or a local model via Ollama**. You can use agent frameworks like if (LangChain agents) you have previous knowledge but it is not mandatory or expected to be used, you can go through the project without those frameworks.

---

2. The Task: "Smart Document Q&A and Summarizer Tool"

Build a small backend application that:

1. **Accepts a document** (PDF or .txt file) from a user.
2. **Splits the document into chunks** and stores them (in memory or a local file/SQLite — no need for a vector database, though you may use one like ChromaDB/FAISS if you want to go further).
3. Lets the user **ask a question about the document**, and the app:
    - ○ Finds the most relevant chunk(s) of text (simple keyword search or embedding similarity — your choice).
    - ○ Sends the question + relevant chunk(s) to an LLM (Groq/Gemini/Ollama).
    - ○ Returns an answer **grounded in the document**, not the LLM's general knowledge.
4. Also provides a **"Summarize this document" endpoint** that returns a concise summary.

5. Has **basic error handling** — e.g., what happens if the file is empty, too large, or not a valid PDF? What happens if the LLM API fails or times out?

This is a simplified RAG (Retrieval-Augmented Generation) task. You do not need fancy retrieval — a working, well-explained simple version is completely acceptable. What matters is that you understand and can explain **why** you built it the way you did.

---

3. Step-by-Step Plan

Phase 1 — Research

- Read about what **RAG (Retrieval-Augmented Generation)** is and why grounding matters.
- Look at 2–3 available ways to chunk text and use the better one for your approach.
- Decide the type of retrieval for your use case. Choose one; be ready to explain your choice and its tradeoffs.
- Pick your LLM provider (Groq / Gemini free tier / Ollama) and get API access or local setup working.
- **Deliverable:** A short `notes.md` or section in your README explaining your design decisions and what you considered/rejected and why.

Phase 2 — Build

- Set up a Python backend using **FastAPI** (you've used this before, so we're testing depth here, not learning a new framework).
- Implement:
    - `POST /upload` — upload a document
    - `POST /ask` — ask a question about the uploaded document
    - `GET /summarize` — get a summary of the uploaded document
- Add input validation (file type, size limits, empty question, etc.) ● Add proper error responses (don't let the app crash on bad input).
- Provide proper response if the query asked by the user does not exist in the file they have uploaded. (handle cases where query may be out of context)
- Write clean, modular code (don't put everything in one file — separate concerns: file handling, chunking, retrieval, LLM calls).

Phase 3 — Test & Document Results

- Test with at least 3 different documents (try a short one, a long one, and a tricky one —

e.g. a scanned/image-heavy PDF, to see how your app handles failure).

- For each test, record: the question asked, the answer received, and whether it was correct/grounded.
- Write all of this into a `test_results.md` file (see Section 5 below for exact format).

Phase 4 — Packaging & Submission

- Clean up the repo, write the README, add a Dockerfile.
- Make sure someone who has never seen your code can clone the repo and run it with minimal steps.
- Submit your GitHub repo link.

---

4. Repository Requirements (Mandatory)

Your GitHub repo must contain:

File/Folder	Purpose
`README.md`	Project overview, setup instructions, how to run, architecture explanation, design decisions
`requirements.txt`	All Python dependencies with versions
`.gitignore`	Exclude `__pycache__`, `.env`, virtual envs, uploaded test files, etc.
`Dockerfile`	So the project can run with one command (e.g. `docker build -t qa-app . && docker run -p 8000:8000 qa-app`)
`test_results.md`	Documented test runs (see format below)
`app/` (or similar)	Your actual source code, organized into logical modules
`.env.example`	Example env file showing what environment variables are needed (API keys etc.) — **never commit your actual API key**

README.md must include:

1. What the project does (2–3 sentences)

2. Architecture diagram or explanation (even a simple text-based flow is fine: Upload → Chunk → Store → Query → Retrieve → LLM → Answer)
3. Setup instructions (local + Docker)
4. Which LLM provider you used and why
5. Known limitations (be honest — we value this more than pretending it's perfect)

test_results.md format (example):

Test 1: Short text document (research_paper_abstract.txt) **Question:**
What is the main finding of this paper?
Answer received: [paste actual output]
Grounded? Yes — matches the abstract directly
Notes: Response time ~2s

Test 2: Long PDF (50-page report)
Question: What does section 3 recommend?
Answer received: [paste actual output]
Grounded? Partially — missed nuance, see notes
Notes: Chunking missed context near page boundaries

Test 3: Edge case (scanned/image PDF with no extractable text)
What happened: [describe — did it fail gracefully? crash? give wrong answer?] **Fix applied (if any):** [describe]

---

5. Ground Rules

- Work independently — this is an individual evaluation, even though you're both given the same brief.
- You can use AI tools (Claude, ChatGPT, Groq, etc.) to help you code and research — that's expected and normal in this job. What we're evaluating is whether **you** understand what you built, can explain trade-offs, and can debug it when something breaks.
- Ask clarifying questions any time — asking good questions is a positive signal, not a negative one.
- If you get stuck, document what you tried and why it didn't work — partial, well-reasoned progress is valued over a silent black box.